3주차: 이진 탐색과 연결 리스트

정렬된 측정 기록과 연결된 대기열

장치가 측정값을 정렬된 행에 저장합니다. 새 측정값을 찾을 때 모든 기록을 확인하지 않을 수 있을까요? 이어서 대기열의 노드 객체를 유지한 채 순서를 바꿉니다.

이번 주에는 후보 구간으로 값을 탐색하고 노드의 연결을 바꿔 리스트를 뒤집거나 병합합니다.

3주차 문제와 학습 목표

5차시: 행렬·회전 배열 탐색. 6차시: 리스트 뒤집기·정렬 리스트 병합.

- 후보 구간과 노드 참조 정의
- 제거한 후보와 보존한 노드 설명
- 갱신 추적·종료 근거·비용 분석

실습: LeetCode 74, 33, 206, 21. 예시와 추적 표는 강의용으로 구성했습니다.

닫힌 구간과 Python 인덱스

- 후보 인덱스: `left`부터 `right`까지 양 끝 포함
- 초기 경계: `left=0`, `right=n-1`
- 반복 조건: `left <= right`
- 검사한 중간점 제외: `mid+1` 또는 `mid-1`
- 행·열 변환: 정수 나눗셈 `//`, 나머지 `%`

탐색 구간의 보존과 축소

정렬된 [2,4,7,9], 목표값 7:

구간	중간 인덱스	비교	다음 구간
[0,3]	1	4 < 7	[2,3]
[2,3]	2	7 = 7	발견

불변식: 목표값이 있다면 [left,right] 안에 존재. 비교가 실패할 때마다 중간점과 한쪽 구간을 제거합니다. left > right이면 남은 후보가 없습니다.

74 · 문제와 상태

LeetCode 74: Search a 2D Matrix · Medium

행렬에서 target의 존재 여부를 반환합니다.

`matrix=[[1,4,7],[9,12,18]], target=12 → True`

[left,right]를 닫힌 후보 구간으로 두고 mid//C, mid%C로 값을 읽습니다.

행렬의 평탄 인덱스

열이 세 개인 행렬에서 평탄 인덱스 4는 행 `4 // 3 = 1`, 열 `4 % 3 = 1`에 해당합니다. 행렬은 그대로 두고 평탄 리스트를 생성하지 않습니다.

각 행의 정렬만으로는 부족합니다. 각 행의 첫 값이 이전 행의 마지막 값보다 커야 전체 순서가 정렬됩니다.

상태 추적

left,right	mid	값	target=12의 다음 범위
0,5	2	7	[3,5]
3,5	4	12	발견

각 행에서 사라진 후보, 보존한 상태, 다음에 읽을 위치를 설명합니다.

행렬 탐색 구현

```
def search_matrix(matrix, target):
    rows, cols = len(matrix), len(matrix[0])
    left, right = 0, rows * cols - 1
    while left <= right:
        mid = (left + right) // 2
        value = matrix[mid // cols][mid % cols]
        if value == target:
            return True
        if value < target:
            left = mid + 1
        else:
            right = mid - 1
    return False
```

74 · 정당성과 비용

행 사이 조건까지 있으므로 평탄 배열은 정렬되어 있습니다. mid가 target보다 작으면 mid 이하를 제거합니다. 매 반복 구간이 줄고, 종료하면 후보가 없습니다.

R,C=행·열 수. O(log(RC)) 시간, O(1) 보조 공간. 평탄 복사하지 않습니다.

33 · 문제와 상태

LeetCode 33: Search in Rotated Sorted Array · Medium

회전된 정렬 배열에서 target의 위치를 반환하고 없으면 -1입니다.

`nums=[6,8,1,3,5], target=3 → 3`

정렬된 절반을 식별하고 target이 그 범위에 있는지로 남길 절반을 정합니다.

회전 배열의 정렬된 절반

[6,8,1,3,5]의 중간값은 1입니다. 오른쪽 절반 [1,3,5]은 정렬되어 있고 목표값 3을 포함하므로 왼쪽 절반과 확인한 중간값을 제거합니다.

서로 다른 값으로 구성된 회전 배열에서는 적어도 한쪽 절반이 정렬됩니다. 그 절반의 양 끝값과 목표값을 비교해 남길 구간을 정합니다.

회전 배열 탐색 구현

```
def search_rotated(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        if nums[left] <= nums[mid]:
            if nums[left] <= target < nums[mid]:
                right = mid - 1
            else:
                left = mid + 1
        else:
            if nums[mid] < target <= nums[right]:
                left = mid + 1
            else:
                right = mid - 1
    return -1
```

33 · 정당성과 비용

서로 다른 값의 한 번 회전 배열에서는 적어도 한 절반이 정렬되어 있습니다. 그 범위 안/밖 검사는 target을 포함할 수 없는 절반만 제거하며 mid±1로 진전합니다.

n=길이. O(log n) 시간, O(1) 보조 공간.

실패 입력과 이해 확인

left=mid는 길이 2 구간에서 정체할 수 있습니다. [[1,9],[2,10]]은 행만 정렬된 다른 문제입니다.

최소 feasible 탐색도 같은 종료 의미인가요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

6차시 · 노드 참조와 연결 갱신

대기열의 기록 객체를 새로 만들지 않고 순서를 바꿉니다. 기존 노드를 재사용하면서 next 참조를 바꿉니다. 다음 노드의 참조를 먼저 보관한 뒤 현재 연결을 뒤집는 순서를 추적합니다.

노드 생성과 참조

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
```

`ListNode(4)`: 노드 생성. `self.val`: 저장한 값. `self.next`: 다음 노드 참조. `None`: 연결의 끝.

참조와 노드의 동일성

노드는 값과 다음 노드를 가리키는 참조를 저장합니다. `a = b`는 같은 객체에 이름 두 개를 붙이므로 `a.next`를 바꾸면 그 객체의 연결이 바뀝니다.

값이 같아도 서로 다른 노드일 수 있습니다. `None`은 단일 연결 리스트의 끝을 나타냅니다.

206 · 문제와 상태

LeetCode 206: Reverse Linked List · Easy

단일 연결 리스트를 뒤집고 새 head를 반환합니다.

`head=[4,6,9] → [9,6,4]`

following에 다음 참조를 보존하고 current.next=previous로 뒤집은 뒤 두 참조를 전진합니다.

연결 하나 뒤집기

`4 → 6 → 9`에서 `current.next = previous`를 실행하기 전에 `following = current.next`로 다음 노드를 보관합니다. 이후 두 참조를 전진합니다.

`previous`는 `None`에서 시작하며 `current`가 `None`에 도달하면 새 head가 됩니다. 기존 노드를 재사용합니다.

상태 추적

단계	previous	current	보존한 following
시작	None	4→6→9	미정
4 처리	4→None	6→9	6
6 처리	6→4	9	9
9 처리	9→6→4	None	None

각 단계에서 노드가 미처리 접미부에서 역순 접두부로 이동하는 과정을 설명합니다.

리스트 역순 구현

```
def reverse_list(head):
    previous, current = None, head
    while current:
        following = current.next
        current.next = previous
        previous, current = current, following
    return previous
```

206 · 정당성과 비용

반복 시작 때 previous는 처리한 접두부의 역순, current는 미처리 접미부입니다. 한 노드를 옮겨도 둘의 합은 원래 노드 전체입니다. current가 None이면 previous가 완성된 역순입니다.

n=노드 수. O(n) 시간, O(1) 보조 공간. 노드를 재사용합니다.

21 · 문제와 상태

LeetCode 21: Merge Two Sorted Lists · Easy

정렬된 두 연결 리스트의 노드를 이어 하나의 정렬 리스트로 반환합니다.

```
first=[1,5], second=[2,3] → [1,2,3,5]
```

dummy 뒤 tail을 유지하고 작은 head를 연결합니다. 한 리스트가 끝나면 나머지를 붙입니다.

정렬된 리스트 병합

`1 → 5`와 `2 → 3`의 head 중 작은 노드를 tail에 연결합니다. 1, 2, 3 순서로 선택한 뒤 남은 5를 붙입니다.

dummy 노드를 사용하면 첫 연결과 이후 연결에 같은 규칙을 적용합니다. 입력 노드가 아닌 dummy 대신 `dummy.next`를 반환합니다.

정렬 리스트 병합 구현

```
def merge_lists(first, second):
    dummy = tail = ListNode()
    while first and second:
        if first.val <= second.val:
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first or second
    return dummy.next
```

21 · 정당성과 비용

tail까지는 정렬된 병합 접두부입니다. 두 head 중 작은 것은 남은 모든 값 중 최소이므로 다음 노드로 안전합니다. 남은 접미부도 정렬되어 있습니다.

n,m=두 노드 수. O(n+m) 시간, O(1) 보조 공간. 입력 연결을 변경합니다.

실패 입력과 이해 확인

다음 참조를 먼저 저장하지 않으면 나머지 연결을 잃습니다. 같은 값의 두 노드도 서로 다른 객체입니다.

같은 값의 노드 두 개는 같은 방문 기록인가요?

교사용 답은 별도 진행안에 있습니다. 코드 설명은 읽기자료의 구현과 줄별로 대조합니다.

기준 풀이와 비용 비교

문제	기준 풀이	이번 풀이
행렬 탐색	전체 순회: O(RC)	인덱스 탐색: O(log(RC))
회전 배열 탐색	전체 순회: O(n)	정렬 절반 탐색: O(log n)
리스트 뒤집기	노드 저장: O(n) 추가 공간	연결 변경: O(1) 추가 공간
리스트 병합	노드 수집·정렬: O(N log N) 시간	병합: O(N) 시간

N=n+m. 정수 비교와 참조 연산은 상수 비용으로 가정합니다. 입력 변환·테스트의 저장 공간은 알고리즘 보조 공간과 별도로 계산합니다.

독립 실습과 참고자료

해설을 읽기 전에 `student_week_03.py`를 구현합니다. 상태 정의·정당성·비용 분석·경계 입력을 함께 제출합니다.

학생 실습 · 풀이 읽기자료 · 교사용 진행안

3주차 코드: `python verify.py` 실행. 직접 구현한 코드는 `--student` 옵션으로 확인합니다.

Erik Demaine·Srini Devadas, MIT 6.006: Introduction to Algorithms (2011).